

CSE 4101: Compiler Design Assignment

Sci-PL: A Mini Compiler for Scientific Computation

Assignment Submission

November 14, 2025

1 Question 1: Lexical Analysis

This section presents the lexical analysis for the provided Sci-PL source code. The following table enumerates all lexemes identified in the source program, their corresponding token types, and the regular expression patterns that define them, as required by the prompt. The table uses alternating row colors for readability.

	Pattern (Regular Expression)	Token Type	Lexeme
main	KW_MAIN	main	
def	KW_DEF	def	
if	KW_IF	if	
else	KW_ELSE	else	
while	KW_WHILE	while	
check_balance	KW_FUNC_CHECK	check_balance	
int	TYPE_INT	int	
float	TYPE_FLOAT	float	
bool	TYPE_BOOL	bool	
reaction	TYPE_REACTION	reaction	
[a-zA-Z_][a-zA-Z0-9_]	IDENTIFIER	x, y, isBalanced, r1, r2	
[0-9]	LITERAL_INT	10, 1, 5, 2	
[0-9][0-9]	LITERAL_FLOAT	5.5, 1.0	
"[^"]"	LITERAL_STRING	"2H2+O2->2H2O", "H2+O2->H2O"	
=	OP_ASSIGN	=	
	OP_PLUS	+	
-	OP_MINUS	-	
	OP_MULTIPLY		
>	OP_GT	>	
<	OP_LT	<	
&&	OP_LOGIC_AND	&&	
{	DELIM_LBRACE	{	
}	DELIM_RBRACE	}	
(	DELIM_LPAREN	(	
)	DELIM_RPAREN	)	
;	DELIM_SEMICOLON	;	

	Pattern (Regular Expression)	Token Type	Lexeme
,	DELIM_COMMA		,

2 Question 2: Syntax Analysis

This section addresses the syntax analysis phase. We provide the Context-Free Grammar (CFG) and the Abstract Syntax Tree (AST) as requested.

2.1 2.1 Context-Free Grammar (CFG)

A simplified Context-Free Grammar (CFG) for the Sci-PL language is presented below.

Context-Free Grammar (CFG) for Sci-PL

$Program \rightarrow KW_MAIN \ DELIM_LBRACE \ StmtList \ DELIM_RBRACE$

$StmtList \rightarrow Stmt \ StmtList \mid \varepsilon$

$Stmt \rightarrow DeclStmt$
 $\mid AssignStmt$
 $\mid IfStmt$
 $\mid WhileStmt$

$DeclStmt \rightarrow KW_DEF \ Type \ IdList \ DELIM_SEMICOLON$

$Type \rightarrow TYPE_INT \mid TYPE_FLOAT \mid TYPE_BOOL \mid TYPE_REACTION$

$IdList \rightarrow IDENTIFIER \ DELIM_COMMA \ IdList \mid IDENTIFIER$

$AssignStmt \rightarrow IDENTIFIER \ OP_ASSIGN \ Expr \ DELIM_SEMICOLON$

$IfStmt \rightarrow KW_IF \ (Expr) \ \{ \ StmtList \} \ KW_ELSE \ \{ \ StmtList \}$
 $\mid KW_IF \ (Expr) \ \{ \ StmtList \}$

$WhileStmt \rightarrow KW_WHILE \ (Expr) \ \{ \ StmtList \}$

$Expr \rightarrow LogicExpr$

$LogicExpr \rightarrow LogicExpr \ OP_LOGIC_AND \ RelExpr \mid RelExpr$

$RelExpr \rightarrow ArithExpr \ OP_GT \ ArithExpr$
 $\mid ArithExpr \ OP_LT \ ArithExpr$
 $\mid ArithExpr$

$ArithExpr \rightarrow ArithExpr \ OP_PLUS \ Term \mid ArithExpr \ OP_MINUS \ Term \mid Term$

$Term \rightarrow Term \ OP_MULTIPLY \ Factor \mid Factor$

$Factor \rightarrow Primary$

$Primary \rightarrow IDENTIFIER \mid LITERAL_INT \mid LITERAL_FLOAT$
 $\mid LITERAL_STRING \mid CallExpr \mid DELIM_LPAREN \ Expr \ DELIM_RPAREN$

$CallExpr \rightarrow KW_FUNC_CHECK \ (Expr)$

1. Parse Tree (CST)

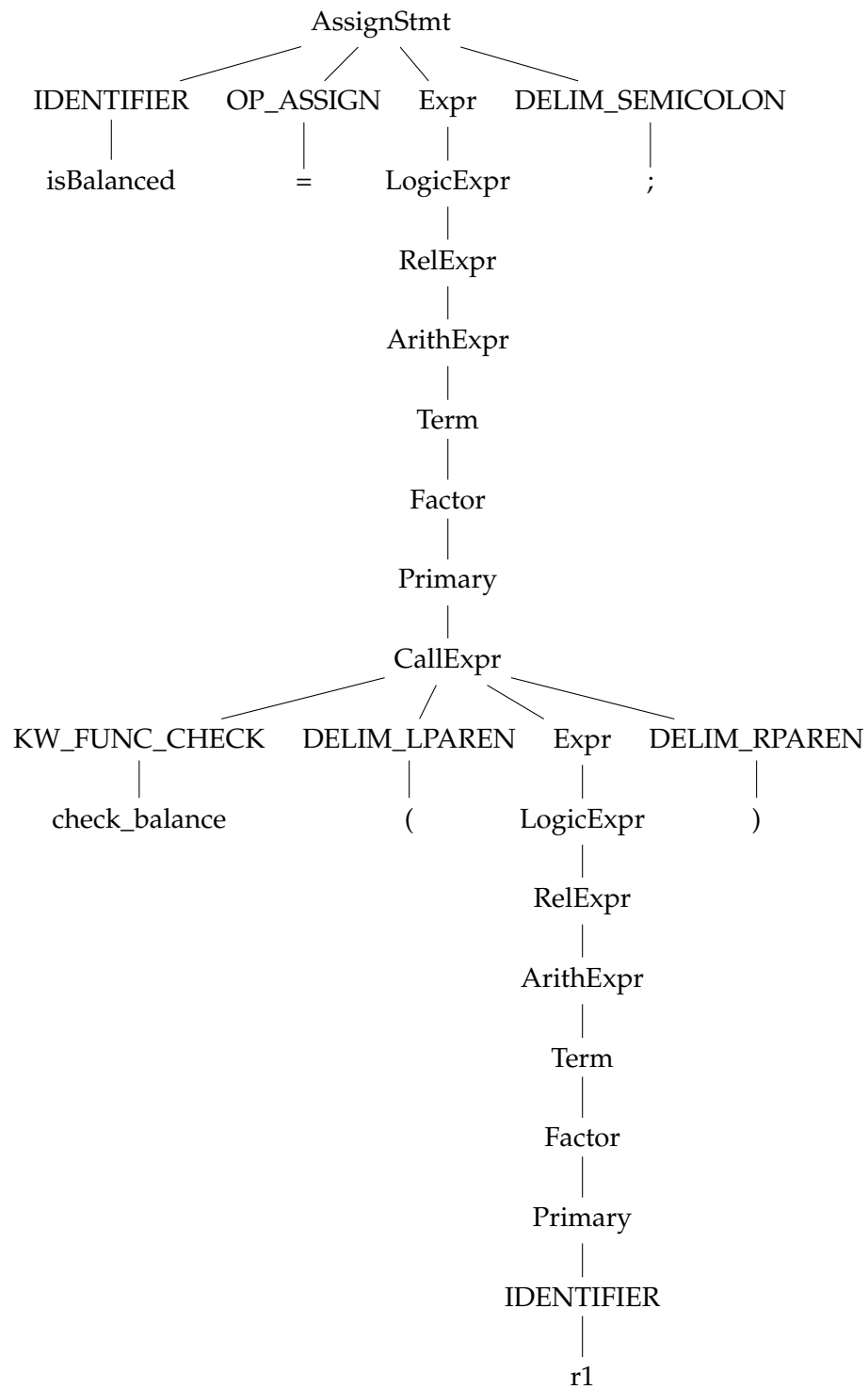


Figure 1: Parse Tree (CST) showing the full syntactic derivation.

2. Abstract Syntax Tree (AST)

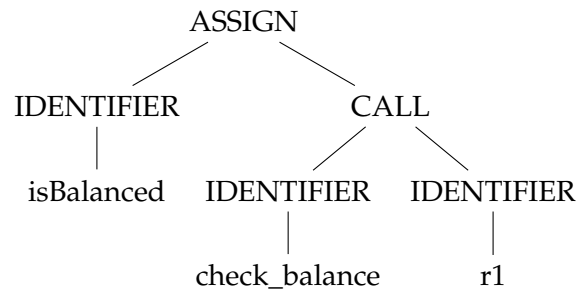


Figure 2: Abstract Syntax Tree (AST) showing the semantic structure.

3 Question 3: Intermediate Code Generation

This section provides the Three-Address Code (TAC) for the main operations of the Sci-PL source program. Temporary variables (e.g., t1, t2) are used to hold intermediate results from expressions, and labels (e.g., L1, L2) are used to manage the control flow for conditional and loop structures. Comments are prepended with #.

This translation explicitly handles the automatic type promotion from int to float specified in the language overview, as seen in operations like `y = x + 5.5;`.

```
1  --- Variable Declarations ---
2  # Declarations (def) are processed by the symbol table. No TAC is generated.
3
4  --- Assignment Statements ---
5  # x = 10;
6  x = 10
7
8  # y = x + 5.5;
9  # Requires type promotion (int -> float)
10 t1 = (float) x
11 t2 = t1 + 5.5
12 y = t2
13
14 # r1 = "2H2+O2->2H2O";
15 r1 = "2H2+O2->2H2O"
16
17 # r2 = "H2+O2->H2O";
18 r2 = "H2+O2->H2O"
19
20 # isBalanced = check_balance(r1);
21 # Follows example: t2 = CALL check_balance(r1)
22 t3 = CALL check_balance(r1)
23 isBalanced = t3
24
25 --- If-Else Control Structure ---
26 # if (check_balance(r2)) {... } else {... }
27 t4 = CALL check_balance(r2)
28 GOTO_IF_FALSE t4 L1
29
```

```

30 # True block
31 # x = x + 1;
32 t5 = x + 1
33 x = t5
34 GOTO L2
35
36 L1: # Else block
37 # x = x - 1;
38 t6 = x - 1
39 x = t6
40
41 L2: # End if-else
42
43 --- While Loop Control Structure ---
44 # while (x > 5) {... }
45 L3: # Loop start
46 t7 = x > 5
47 GOTO_IF_FALSE t7 L4
48
49 # Loop body
50 # y = y + 1.0;
51 t8 = y + 1.0
52 y = t8
53
54 # x = x - 1;
55 t9 = x - 1
56 x = t9
57 GOTO L3
58
59 L4: # End while
60
61 --- If Control Structure ---
62 # if (isBalanced && (x < 10)) {... }
63 t10 = x < 10
64 t11 = isBalanced && t10
65 GOTO_IF_FALSE t11 L5
66
67 # True block
68 # y = y * 2;
69 # Requires type promotion (int -> float)
70 t12 = (float) 2
71 t13 = y * t12
72 y = t13
73
74 L5: # End if

```

4 Question 4: Assembly Code Generation

Finally, this section provides a translation of the Three-Address Code from Section 3 into a MIPS-like pseudo-assembly language. We assume a load/store architecture. Variables are stored in a .data segment and loaded into registers for operations.

- Integer values use general-purpose registers (e.g., \$t0).
- Floating-point values use floating-point registers (e.g., \$f0).
- Function arguments are passed in \$a0 and return values in \$v0.

```

1  .data
2  # Variable storage
3  x:          .word    0
4  y:          .float   0.0
5  isBalanced: .word    0
6  r1:         .string  "2H2+O2->2H2O"
7  r2:         .string  "H2+O2->H2O"
8
9  # Constants used in the code
10 const_5_5:   .float   5.5
11 const_1_0:   .float   1.0
12 const_2_0:   .float   2.0
13 const_int_1: .word    1
14 const_int_2: .word    2
15 const_int_5: .word    5
16 const_int_10: .word   10
17
18 .text
19 .globl main
20 main:
21 # x = 10; (TAC: x = 10)
22 LI $t0, 10          # t0 = 10
23 STORE $t0, x        # x = t0
24
25 # y = x + 5.5; (TAC: t1 = (float)x, t2 = t1 + 5.5, y = t2)
26 LOAD $t0, x         # t0 = x (int)
27 CVT_I_F $f0, $t0    # f0 = (float)t0 (t1)
28 LOAD_FLOAT $f1, const_5_5 # f1 = 5.5
29 ADD_FLOAT $f2, $f0, $f1 # f2 = f0 + f1 (t2)
30 STORE_FLOAT $f2, y   # y = f2
31
32 # r1, r2 assignments are handled by .data initialization
33
34 # isBalanced = check_balance(r1); (TAC: t3 = CALL..., isBalanced = t3)
35 LOAD_ADDR $a0, r1    # a0 = address of r1
36 JAL check_balance    # Call function
37 # v0 holds return value (t3)
38 STORE $v0, isBalanced # isBalanced = v0
39
40 # if (check_balance(r2)) {... } else {... }
41 # (TAC: t4 = CALL..., GOTO_IF_FALSE t4 L1)
42 LOAD_ADDR $a0, r2    # a0 = address of r2
43 JAL check_balance    # Call function
44 # v0 holds return value (t4)
45 BEQ $v0, $zero, L1   # if (v0 == 0) goto L1
46
47 # True block: x = x + 1; (TAC: t5 = x + 1, x = t5)
48 LOAD $t0, x

```

```

49 ADDI $t1, $t0, 1      # t1 = t0 + 1 (t5)
50 STORE $t1, x
51 J L2                  # Skip else block
52
53 L1: # Else block: x = x - 1; (TAC: t6 = x - 1, x = t6)
54 LOAD $t0, x
55 SUBI $t1, $t0, 1      # t1 = t0 - 1 (t6)
56 STORE $t1, x
57
58 L2: # End if-else
59
60 # while (x > 5) {...} (TAC: L3:, t7 = x > 5, GOTO_IF_FALSE t7 L4)
61 L3:
62 LOAD $t0, x           # t0 = x
63 LI $t1, 5             # t1 = 5
64 SGT $t7, $t0, $t1     # t7 = (x > 5)
65 BEQ $t7, $zero, L4    # if (t7 == 0) goto L4
66
67 # Loop body: y = y + 1.0; (TAC: t8 = y + 1.0, y = t8)
68 LOAD_FLOAT $f0, y      # f0 = y
69 LOAD_FLOAT $f1, const_1_0 # f1 = 1.0
70 ADD_FLOAT $f2, $f0, $f1 # f2 = y + 1.0 (t8)
71 STORE_FLOAT $f2, y
72
73 # x = x - 1; (TAC: t9 = x - 1, x = t9)
74 LOAD $t0, x
75 SUBI $t1, $t0, 1      # t1 = t0 - 1 (t9)
76 STORE $t1, x
77 J L3                  # Jump to loop start
78
79 L4: # End while
80
81 # if (isBalanced && (x < 10)) {...}
82 # (TAC: t10 = x < 10, t11 = isBalanced && t10, GOTO_IF_FALSE t11 L5)
83
84 # Check first condition: isBalanced
85 LOAD $t0, isBalanced   # t0 = isBalanced
86 BEQ $t0, $zero, L5     # Short-circuit: if (!isBalanced) goto L5
87
88 # Check second condition: x < 10
89 LOAD $t1, x            # t1 = x
90 LI $t2, 10             # t2 = 10
91 SLT $t10, $t1, $t2     # t10 = (x < 10)
92
93 # t11 = t0 && t10
94 AND $t11, $t0, $t10
95 BEQ $t11, $zero, L5    # if (t11 == 0) goto L5
96
97 # True block: y = y * 2;
98 # (TAC: t12 = (float)2, t13 = y * t12, y = t13)
99 LOAD_FLOAT $f0, y
100 LOAD_FLOAT $f1, const_2_0 # f1 = 2.0 (t12)
101 MUL_FLOAT $f2, $f0, $f1 # f2 = y * 2.0 (t13)
102 STORE_FLOAT $f2, y

```

```
103  
104 L5: # End if  
105  
106 # End of main program  
107 LI $v0, 10          # Syscall for exit  
108 SYSCALL
```